

CS 486/686

Reinforcement Learning

Yuntian Deng

Lecture 15

RN 22.1–22.3 · PM 12.1, 12.5, 12.8

Heads-up: next week is asynchronous

 I'll be away at ICML the week of July 7. **L17 (Tue Jul 7)** and **L18 (Thu Jul 9)** will be **pre-recorded videos** — no in-person class those days. Watch on your own time (links posted before class).

 **Deadlines are unchanged:** Chat 8 + the CS686 project proposal are due **Tue Jul 7**; Chat 9 is out and **Assignment 2 is due Thu Jul 9**.

 Questions? Post on **Piazza** — the TAs are available all week.

Search ›

Uncertainty ›

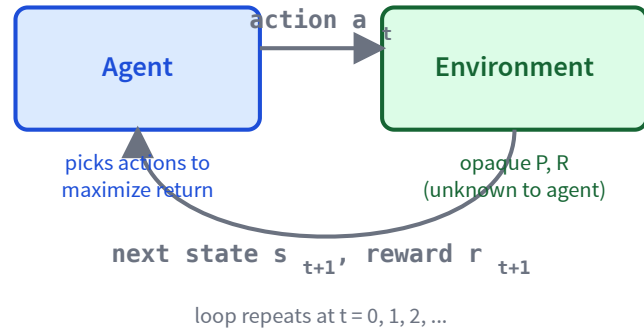
Decisions ›

Learning

Learning goals

- Trace + implement **passive adaptive dynamic programming** (passive ADP).
- Explain the **exploration vs exploitation** trade-off.
- Trace + implement **active ADP**.
- Trace + implement **Q-learning**.
- Trace + implement **SARSA**.
- Tell **on-policy** from **off-policy** learning.

The RL setting: no P , no R up front



An MDP whose **transition P** and **reward R** are unknown. The agent sees state s and reward r , picks an action a , and observes the next state s' .

Goal: maximize discounted return — while learning the world as it goes.

Three new challenges in RL

Credit assignment

A reward now may be due to an action many steps ago. Which one gets the credit?

Long-term impact

An action shapes not just the next reward, but every future state and reward.

Explore vs exploit

Try new actions to learn more, or cash in on the best known so far?

Passive learning: just evaluate a fixed policy

Fix a policy π . Goal: learn $V^\pi(s)$ for every state s .

Adaptive Dynamic Programming (ADP)

1. **Learn** the transition $P(s' \mid s, a)$ and reward $R(s)$ from observed experience.
2. **Solve** the Bellman policy-evaluation equations:

$$V^\pi(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) V^\pi(s').$$

A model-based approach — it builds an explicit model of the world.

The passive ADP algorithm

passive-ADP(π)

1. Initialize $R(s)$, $N(s, a)$, $N(s, a, s')$, $V^\pi(s)$.
2. Follow π and observe an experience $\langle s, a, s', r \rangle$.
3. Update reward: $R(s) \leftarrow r$.
4. Update counts and transition probability:

$$N(s, a) += 1, \quad N(s, a, s') += 1, \quad P(s' \mid s, a) = \frac{N(s, a, s')}{N(s, a)}.$$

5. Re-solve Bellman to get $V^\pi(s)$ (exactly or iteratively).

The estimate $P(s' \mid s, a) = \frac{N(s, a, s')}{N(s, a)}$ is just the **maximum-likelihood** fraction: how often a in s led to s' . Each Bellman re-solve costs $O(n^3)$ (or a few iterations).

Passive ADP: update the model

s_{11}	+1
s_{21}	-1

$\pi(s_{11}) = \text{down}, \pi(s_{21}) = \text{right}, \gamma = 0.9.$

Before: $N(s, a) = 5; N(s, a, s') = 3$ intended, 1 to each side.

New experience: $\langle s_{11}, \text{down}, s_{21}, -0.04 \rangle$.

Increment counts: $N(s_{11}, \text{down}): 5 \rightarrow 6,$

$N(s_{11}, \text{down}, s_{21}): 3 \rightarrow 4.$

Re-estimate $P(\cdot \mid s_{11}, \text{down})$ as count fractions:

$$s_{21}: \frac{4}{6}, \quad \text{stay } s_{11}: \frac{1}{6}, \quad +1: \frac{1}{6}$$

(Going down can slip sideways: left bumps the wall $\rightarrow$ stay; right $\rightarrow$ the +1 cell.)

Passive ADP: re-solve with the new model

Plug the estimated probabilities into the policy-evaluation equations ($R = -0.04$, $\gamma = 0.9$). For s_{21} , $\pi(s_{21}) = \text{right}$: $\frac{3}{5}$ to -1 , $\frac{1}{5}$ up to s_{11} , $\frac{1}{5}$ stay.

$$\begin{aligned} V(s_{11}) &= -0.04 + 0.9 \left[\frac{4}{6} V(s_{21}) + \frac{1}{6} V(s_{11}) + \frac{1}{6} (+1) \right] \\ V(s_{21}) &= -0.04 + 0.9 \left[\frac{3}{5} (-1) + \frac{1}{5} V(s_{11}) + \frac{1}{5} V(s_{21}) \right] \end{aligned}$$

Two linear equations, two unknowns — solve:

$$V(s_{11}) = -0.438, \quad V(s_{21}) = -0.803.$$

Every experience refines the model; re-solving Bellman propagates it to all states' values.

Quick check: probability update

Q1. In a 3x4 grid, suppose $N(s_{23}, \text{down}) = 23$ and $N(s_{23}, \text{down}, s_{24}) = 2$. The agent tries to move *down* but slips and lands in s_{24} . What is the updated $P(s_{24} \mid s_{23}, \text{down})$?

- A. $2/23 \approx 0.087$
- B. $3/23 \approx 0.130$
- C. $3/24 = 0.125$
- D. $2/24 \approx 0.083$

C — 0.125. First increment the counts: $N(s_{23}, \text{down}) \rightarrow 24$ and $N(s_{23}, \text{down}, s_{24}) \rightarrow 3$. Then $P = 3/24 = 0.125$.

Active learning: explore vs exploit

Now the agent *chooses* its actions. Two competing pressures:

Exploit

Take the action that maximizes the current $V(s)$ (or $Q(s, a)$). Reap known rewards.

Explore

Try a different action. Gather data to learn a better model / better Q-values.

A purely greedy agent *seldom* converges to the optimal policy — the learned model is biased toward states the agent has already preferred.

Exploration 1: ϵ -greedy



action probability

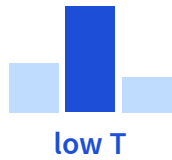
With probability ϵ : take a **random** action (explore).

With probability $1 - \epsilon$: take the **greedy** action (exploit).

Start with a large ϵ and **decay** it over time — explore early, exploit once the values settle.

Simple and popular, but it treats every sub-optimal action as equally worth trying.

Exploration 2: softmax (Boltzmann)



Pick actions in proportion to their value, tuned by a **temperature** T :

$$P(a) = \frac{e^{Q(s,a)/T}}{\sum_{a'} e^{Q(s,a')/T}}$$

- Low T : nearly **greedy** (peaked on the best action).
- High T : nearly **uniform** (lots of exploration).

Unlike ϵ -greedy, better actions are explored more than clearly-bad ones. Cool T over time (like simulated annealing).

Exploration 3: optimistic initialization



Assume the best about actions you haven't tried enough. Replace a value u with an exploration function f :

$$f(u, n) = \begin{cases} R^+ & n < N_e \\ u & \text{otherwise} \end{cases}$$

R^+ is an optimistic reward bound; N_e is how many tries before we trust the real estimate u .

Every action looks great until tried N_e times, pulling the agent toward the unknown.

Active ADP: bake exploration into the value

Active ADP = passive ADP, but now the agent *chooses* actions. The trick: act greedily on an **optimistic** value V^+ , seen through the exploration function f from before.

Pick the action whose successor value — *viewed through f* — looks best:

$$a = \arg \max_a f \left(\underbrace{\sum_{s'} P(s' \mid s, a) V^+(s')}_{\text{estimated value of } a}, \underbrace{N(s, a)}_{\text{times tried}} \right)$$

f inflates the value of **under-tried** actions ($N(s, a) < N_e$), so the agent is pulled toward exploring them.

The active ADP algorithm

active-ADP(N_e, R^+)

1. Initialize $R(s)$, $V^+(s)$, $N(s, a)$, $N(s, a, s')$.
2. Repeat until every (s, a) is tried $\geq N_e$ times and V^+ converges:
 - Pick $a = \arg \max_a f(\sum_{s'} P(s' | s, a) V^+(s'), N(s, a))$ (the rule above).
 - Execute a ; observe $\langle s, a, s', r \rangle$; update $R(s) \leftarrow r$, counts, and $P(s' | s, a)$.
 - $V^+(s) \leftarrow R(s) + \gamma \max_a f(\sum_{s'} P(s' | s, a) V^+(s'), N(s, a))$.
 - $s \leftarrow s'$.

Same model-based loop as passive ADP — only the action choice and the **max** now pass through f .

From V to Q: learn without a model

Bellman for V and Q are both fixed-point equations:

Bellman for V^*

$$V^*(s) = R(s) + \gamma \max_a \sum_{s'} P(s' | s, a) V^*(s')$$

To pick π^* you still need to know P and combine it with V^* .

Bellman for Q^*

$$Q^*(s, a) = R(s) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a')$$

If we know Q^* , then $\pi^*(s) = \arg \max_a Q^*(s, a)$ — no P needed.

model-free!

Where does Q^* come from? Plug $V^* = \max_{a'} Q^*$ into $Q^* = R + \gamma \sum P V^*$.

Learning Q directly is what makes RL practical at scale.

Temporal difference (TD) error

After one transition $\langle s, a, r, s' \rangle$, pretend that transition was certain ($P(s' | s, a) = 1$). Then by Bellman:

$$Q(s, a) \stackrel{?}{=} \underbrace{R(s) + \gamma \max_{a'} Q(s', a')}_{\text{target}}$$

TD error

$$\delta = \left(R(s) + \gamma \max_{a'} Q(s', a') \right) - Q(s, a)$$



Idea: nudge $Q(s, a)$ toward the target by an amount proportional to δ .

The Q-learning algorithm

Q-learning(α, γ)

1. Initialize $Q(s, a)$ arbitrarily, $R(s)$, $N(s, a)$.
2. Repeat until Q converges:
 - Pick a from s using *any* exploration strategy (ϵ -greedy, softmax, optimistic).
 - Execute a ; observe $\langle s, a, s', r \rangle$; update $R(s) \leftarrow r$.
 - $Q(s, a) \leftarrow Q(s, a) + \alpha \underbrace{\left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)}_{\delta, \text{ the TD error}}.$
 - $s \leftarrow s'.$

- The update is exactly $Q \leftarrow Q + \alpha \delta$ — nudge Q a fraction α of the way toward the target.
- **Learning rate** $\alpha(N) = 10/(9 + N(s, a))$ shrinks with visits: early surprises move Q a lot, later ones fine-tune.
- **Model-free**; converges to Q^* if every (s, a) is visited infinitely often.

Q-learning: a single update

3x4 grid. Experience: $\langle s_{32}, \text{right}, s_{33}, -0.04 \rangle$. Take $\gamma = 1, \alpha = 0.1$.

	s_{32}	s_{33}
up	0.55	0.4
right	0.7	0.9
down	0.5	0.8
left	0.4	0.5

Yellow cell = the one we update; green = the max-value action in s_{33} .

$$\text{target} = r + \gamma \cdot \max_{a'} Q(s_{33}, a') = -0.04 + 1.0 \cdot 0.9 = 0.86$$

$$\delta = \text{target} - Q(s_{32}, \text{right}) = 0.86 - 0.7 = 0.16$$

$$Q(s_{32}, \text{right}) \leftarrow 0.7 + 0.1 \cdot 0.16 = 0.716$$

SARSA: on-policy temporal difference

Observe a 5-tuple $\langle s, a, r, s', a' \rangle$ where a' is the action **actually taken** in s' (per the current policy):

Q-learning (off-policy)

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Target uses the *greedy* next action. Learns about the optimal policy regardless of what's actually played.

SARSA (on-policy)

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a))$$

Target uses the *actually-played* next action a' . Learns about the policy the agent is following (exploration and all).

SARSA = State, Action, Reward, State, Action.

SARSA vs Q-learning: the scenario

Update $Q(s_{13}, \text{down}) = 0.4$ after $\langle s_{13}, \text{down}, s_{23} \rangle$, with $r = -0.04$, $\gamma = 0.9$, $\alpha = 0.4$.

The ε -greedy step explores $a' = \text{right}$ in s_{23} (into the bad cell).

$Q(s_{23}, \cdot)$	up	right	down	left
	0.3	-0.4	0.7	0.3

Yellow = the explored next action $a' = \text{right}$ (SARSA's target); green = $\max_{a'} Q$ (Q-learning's target).

SARSA vs Q-learning: the two updates

SARSA: uses $a' = \text{right}$

$$\begin{aligned}\text{target} &= -0.04 + 0.9 \cdot (-0.4) = -0.4 \\ \delta &= -0.4 - 0.4 = -0.8 \\ Q(s_{13}, \text{down}) &\leftarrow 0.4 + 0.4 \cdot (-0.8)\end{aligned}$$

= 0.08 (punished for the bad step)

Q-learning: uses $\max_{a'} = 0.7$

$$\begin{aligned}\text{target} &= -0.04 + 0.9 \cdot 0.7 = 0.59 \\ \delta &= 0.59 - 0.4 = 0.19 \\ Q(s_{13}, \text{down}) &\leftarrow 0.4 + 0.4 \cdot 0.19\end{aligned}$$

= 0.476 (rewarded for the best)

SARSA is **conservative** (learns what really happens); Q-learning is **aggressive** (learns the optimal policy).

ADP vs Q-learning vs SARSA

	ADP	Q-learning	SARSA
Learns transition P ?	yes (model-based)	no (model-free)	no (model-free)
On / off policy	n/a (eval-only)	off-policy	on-policy
Computation per experience	heavy (solve Bellman)	cheap (one TD update)	cheap (one TD update)
Convergence speed	fast (few experiences)	slower, higher variance	slower, higher variance

ADP is sample-efficient but expensive per step; TD methods (Q, SARSA) flip that trade-off.

When to use what (Q2)

Q2. For a high-stakes *online* learning problem (for example, a self-driving car still in training), which algorithm is safer?

- A. Q-learning
- B. **SARSA**
- C. I don't know

B — SARSA. SARSA's target uses the action that will *actually* be taken while exploring, so it learns to avoid catastrophic outcomes along the exploration path. Q-learning's target uses the greedy action, ignoring the cost of exploration mistakes.

Putting the labels together (Q3)

Q3. Which statement is true?

- A. Q-learning is model-free + off-policy; SARSA is model-based + on-policy.
- B. Q-learning is model-based + off-policy; SARSA is model-free + on-policy.
- C. Q-learning is model-free + on-policy; SARSA is model-based + off-policy.
- D. Q-learning is model-based + on-policy; SARSA is model-free + off-policy.
- E. **None of the above.**

E — None of the above. Both Q-learning and SARSA are *model-free*. Q-learning is off-policy (target uses $\max_{a'} Q$); SARSA is on-policy (target uses the action actually played).

Learning goals (recap)

- ✓ Trace + implement **passive ADP**.
- ✓ Explain the **exploration vs exploitation** trade-off.
- ✓ Trace + implement **active ADP**.
- ✓ Trace + implement **Q-learning**.
- ✓ Trace + implement **SARSA**.
- ✓ Tell **on-policy** from **off-policy**.

Search ›

Uncertainty ›

Decisions ›

Learning

Next module: Machine Learning and Deep Learning

RL was learning from rewards. The last module turns to *learning from data*.

- **L16** — supervised learning, bias / variance, cross-validation.
- **L17** — unsupervised: k-means, PCA, autoencoders, GANs.
- **L18** — decision trees: entropy, information gain, ID3.
- **L19–L21** — neural networks: forward / backward / optimizers.